

Línea de comandos: explorar, filtrar y verificar archivos

Una guía práctica para el trabajo en datos · Linux, macOS y una terminal Linux en WSL.

Antes de analizar un archivo es necesario conocer su ubicación, su contenido y el cumplimiento de las condiciones esperadas. La guía presenta comandos pequeños que pueden combinarse para explorar, buscar, resumir y verificar los resultados.

1 · Explorar

Ubicar archivos, reconocer su tipo y leer una muestra.

`pwd · ls · head`

2 · Consultar

Buscar coincidencias y trabajar con registros y columnas.

`grep · wc · awk`

3 · Verificar

Detectar faltantes, duplicados y diferencias.

`sort · uniq · diff`

E1 · Preparar el entorno

Rutas, carpetas y archivos

E2 · Inspeccionar un archivo

Tipo, tamaño y contenido

E3 · Buscar y contar

`grep`, `wc` y patrones

E4 · Trabajar con columnas

`AWK`, filtros y cálculos

E5 · Verificar la calidad

Vacíos, duplicados y rangos

E6 · Combinar y conservar

Tuberías, informes e integridad

Cheat sheet · 100 comandos de Linux

Opciones habituales y ejemplos por categoría

Desafío · Práctica integradora

Revisar un archivo y justificar los hallazgos

Cómo usar esta guía. Los bloques muestran comandos listos para copiar, sin el símbolo del prompt. Los pasos de preparación se ejecutan una vez y se conserva la misma terminal. Las salidas esperadas corresponden a los archivos de práctica incluidos.

Los ejemplos usan Bash o Zsh. En Windows se ejecutan dentro de WSL; no se deben copiar directamente a PowerShell. Algunas opciones adicionales cambian entre macOS y Linux: la ayuda local está disponible mediante `man comando`.

La idea central: un comando consulta o transforma una entrada y produce una salida. Una salida breve puede revelar un problema; no demuestra por sí sola que todo el archivo sea correcto.

Preparar el entorno

Se prepara una carpeta de práctica con archivos de contenido conocido.

La **terminal** es la ventana de interacción; el **shell** interpreta las instrucciones introducidas. Un comando suele tener la forma `comando opciones argumentos`. En `ls -lh datos/`, la opción `-l` pide detalles y `-h` presenta tamaños legibles.

Comando o ruta	Para qué sirve
<code>pwd</code> / <code>ls -lah</code>	Ver la ubicación / listar también archivos ocultos.
<code>cd carpeta</code> / <code>cd ..</code>	Entrar a una carpeta / subir un nivel.
<code>cd ~</code> / <code>cd -</code>	Ir a la carpeta personal / volver a la ubicación anterior.
<code>.</code> / <code>..</code> / <code>~</code>	Directorio actual / superior / personal.
<code>mkdir -p datos resultados</code>	Crear carpetas si no existen.

E1 · PASO 1

Crear un espacio de trabajo

```
mkdir practica-cli
cd practica-cli
mkdir datos resultados
pwd
```

Si `practica-cli` ya existe, se elige otro nombre para una práctica nueva. Las rutas con espacios se escriben entre comillas: `cd "Mis documentos"`.

E1 · PASO 2

Crear los datos del ejercicio

```
cat > datos/notas.txt <<'EOF'
id;nombre;grupo;nota
1;Ana;A;4.2
2;Luis;B;3.1
3;Marta;A;4.8
4;Juan;B;
3;Marta;A;4.8
5;Sara;C;6.0
EOF
```

El bloque `<<'EOF'` entrega varias líneas al comando `cat`. La línea final `EOF` debe escribirse sola. En este ejemplo se utiliza punto y coma como separador, punto decimal y una fila de encabezado.

Conservar el original. `>` crea o sobrescribe el destino. Este bloque se ejecuta únicamente dentro de la carpeta de práctica. El archivo contiene deliberadamente una nota vacía, un identificador repetido y una nota fuera del rango 0 a 5.

Inspeccionar un archivo

Se reconoce el contenido antes de filtrarlo o modificarlo.

```
file datos/notas.txt
ls -lh datos/notas.txt
head -n 3 datos/notas.txt
tail -n 2 datos/notas.txt
wc -l datos/notas.txt
```

La muestra inicial permite reconocer el encabezado y el separador. La muestra final puede revelar un archivo incompleto o registros inesperados. El archivo de práctica tiene **7 líneas terminadas en salto de línea: 1 encabezado y 6 filas de datos**.

Comando	Cuándo usarlo
<code>cat archivo.txt</code>	Leer completo un archivo pequeño.
<code>less archivo.txt</code>	Explorar texto largo sin imprimirlo todo en pantalla.
<code>head -n 10 archivo.txt</code>	Leer las diez primeras líneas.
<code>tail -n 10 archivo.txt</code>	Leer las diez últimas líneas.
<code>tail -f proceso.log</code>	Observar nuevas líneas de un registro; salir con Ctrl + C.
<code>nl -ba archivo.txt</code>	Mostrar números de línea, incluidas las vacías.
<code>sed -n '3,6p' archivo.txt</code>	Mostrar únicamente las líneas 3 a 6.

Explorar sin perder la ubicación

Dentro de `less`, se utiliza `/Marta` para buscar, `n` para seguir, `g` para ir al inicio, `G` para ir al final y `q` para salir.

```
find datos -type f -name '*.txt'
du -sh datos
df -h .
```

`find` localiza archivos; las comillas conservan el patrón para que lo interprete ese comando. `du` mide el espacio ocupado y `df` informa sobre el sistema de archivos donde está la carpeta.

Tipo y extensión son pistas distintas. Un nombre terminado en `.csv` no garantiza que el contenido sea CSV. `file` ayuda a reconocer el tipo, pero no valida columnas, reglas del negocio ni todos los registros.

Buscar y contar con grep y wc

Se localizan registros y se cuenta cuántas líneas cumplen una condición.

```
grep -n 'Marta' datos/notas.txt
grep -c ';A;' datos/notas.txt
grep -v '^id;' datos/notas.txt
```

La primera consulta encuentra a Marta en las líneas **4 y 6**. La segunda devuelve **3** líneas del grupo A. La tercera excluye las líneas que comienzan por `id;`. Para omitir exactamente la primera línea, se utiliza `awk 'NR > 1'`.

Opción de grep	Qué cambia
<code>-n</code>	Incluye el número de línea.
<code>-i</code>	Ignora diferencias entre mayúsculas y minúsculas.
<code>-v</code>	Selecciona líneas que no coinciden.
<code>-c</code>	Cuenta líneas coincidentes, no apariciones.
<code>-F</code>	Interpreta la búsqueda como texto literal.
<code>-E</code>	Activa expresiones regulares extendidas.
<code>-r -n</code>	Busca recursivamente en una carpeta y muestra líneas.
<code>-C 2</code>	Incluye dos líneas de contexto alrededor de cada coincidencia.

Patrones que conviene reconocer

```
grep '^id;' datos/notas.txt      # Inicio de línea
grep ';$' datos/notas.txt       # Fin de línea
grep -E 'Ana|Luis' datos/notas.txt # Una alternativa u otra
grep -F '4.8' datos/notas.txt    # Punto literal
grep -n '^[[:space:]]*$' datos/notas.txt
```

El último comando detecta líneas vacías o compuestas solo por espacios y tabulaciones. No muestra resultados en esta muestra: un **campo vacío** dentro de una fila no equivale a una **línea vacía**.

Comando	Qué cuenta
<code>wc -l datos/notas.txt</code>	Saltos de línea: 7 en la muestra.
<code>wc -w datos/notas.txt</code>	Palabras separadas por espacios en blanco; no columnas con punto y coma.
<code>wc -c datos/notas.txt</code>	Bytes.
<code>wc -m datos/notas.txt</code>	Caracteres según la configuración regional.

El último salto importa. `wc -l` no cuenta una última línea sin salto final. `awk 'END {print NR+0}' archivo.txt` cuenta también ese registro. Los bytes y los caracteres pueden diferir en texto con tildes.

Trabajar con AWK

Se seleccionan campos, se aplican condiciones y se calcula un resumen.

AWK procesa registros, normalmente líneas. La forma básica es `awk 'condición {acción}' archivo`. Sin condición, la acción se aplica a todos los registros. Por defecto separa campos por espacios y tabulaciones; en este caso se declara `-F ' ; ' .`

Elemento	Significado
<code>\$0</code>	Registro completo.
<code>\$1</code> , <code>\$2</code> , <code>\$NF</code>	Primer campo, segundo campo y último campo.
<code>NR</code> / <code>NF</code>	Número de registro leído / cantidad de campos del registro.
<code>BEGIN</code> / <code>END</code>	Acciones antes de leer / después de terminar.
<code>&&</code> / <code> </code>	Y lógico / O lógico.

Seleccionar y filtrar

```
# Nombre y nota, sin encabezado
awk -F';' 'NR > 1 {print $2, $4}' datos/notas.txt

# Filas del grupo A
awk -F';' 'NR > 1 && $3 == "A" {print $0}' datos/notas.txt

# Filas con nota presente y dentro del rango esperado
awk -F';' 'NR > 1 && $4 != "" && $4 >= 0 && $4 <= 5 {
    print $2, $4
}' datos/notas.txt
```

El último filtro devuelve Ana, Luis y Marta dos veces. Una condición de rango no elimina duplicados. En este paso se supone que toda nota no vacía tiene formato numérico; en E5 se comprueba también esa condición.

Calcular con una condición explícita

```
awk -F';' 'NR > 1 && $4 != "" && $4 >= 0 && $4 <= 5 {
    suma += $4; n++
}
END {
    if (n > 0) printf "Filas: %d | Promedio: %.3f\n", n, suma/n
    else print "Sin notas válidas"
}' datos/notas.txt
```

Resultado esperado: `Filas: 4 | Promedio: 4.225`, con punto decimal. El promedio incluye el registro repetido; todavía no es un resultado depurado.

Separar campos no equivale a interpretar CSV. `awk -F ' , '` sirve para texto simple separado por comas. Si hay delimitadores entre comillas o saltos dentro de campos, se requiere un lector CSV, por ejemplo el módulo `csv` de Python. Para TSV simple se utiliza `-F '\t'`.

Verificar la estructura y los valores

Se reúne evidencia de los problemas antes de decidir cómo corregirlos.

1. Comprobar columnas y campos obligatorios

```
awk -F';' 'NF != 4 {print "Columnas incorrectas, línea", NR}' datos/notas.txt
awk -F';' 'NR > 1 && $4 ~ /^[[:space:]]*$/ {
    print "Nota vacía, línea", NR
}' datos/notas.txt
```

No hay filas con cantidad incorrecta de columnas. Sí hay una nota vacía en la **línea 5**: la fila conserva sus cuatro campos, aunque el último no tenga contenido.

2. Separar el formato numérico de la regla de rango

```
awk -F';' 'NR > 1 {
    if ($4 ~ /^[[:space:]]*$/ ) next
    if ($4 !~ /^[0-9]+([.][0-9]+)?$/ )
        print "Formato inválido, línea", NR ":", $4
    else if ($4+0 < 0 || $4+0 > 5)
        print "Fuera de rango, línea", NR ":", $4
    }' datos/notas.txt
```

La regla acepta enteros no negativos y decimales con punto, sin espacios ni signo. La muestra produce **Fuera de rango, línea 7: 6.0**. Otros formatos, como `4,2`, deben tratarse mediante una regla distinta y explícita.

3. Detectar duplicados y frecuencias

```
# Identificadores presentes más de una vez
awk -F';' 'NR > 1 {print $1}' datos/notas.txt | sort | uniq -d

# Filas completas repetidas, excluyendo el encabezado
awk 'NR > 1' datos/notas.txt | sort | uniq -d

# Cantidad de filas por grupo
awk -F';' 'NR > 1 {print $3}' datos/notas.txt | sort | uniq -c
```

Los resultados muestran el identificador **3** y la fila `3;Marta;A;4.8` repetidos. Las frecuencias son **A: 3, B: 2, C: 1**. `uniq` compara líneas consecutivas; por eso se ordenan primero.

Detectar antes de eliminar. Dos filas con la misma clave pueden contener datos distintos. `sort -u` elimina líneas idénticas, pero no decide qué registro debe prevalecer cuando una clave tiene versiones diferentes.

Combinar comandos y conservar resultados

Se guarda una salida revisable y se comprueba que el original se conserva.

Operador	Función
	Envía la salida estándar a la entrada del siguiente comando.
> / >>	Sobrescribe el destino / agrega al final.
2>	Guarda los mensajes de error.
&&	Ejecuta el siguiente comando si el anterior terminó con código 0.

```
awk -F';' 'NR > 1 {print $3}' datos/notas.txt |
  sort | uniq -c | sort -nr > resultados/frecuencias.txt
cat resultados/frecuencias.txt
cp -i datos/notas.txt resultados/notas_copia.txt
cmp datos/notas.txt resultados/notas_copia.txt
```

`cmp` no imprime nada cuando los archivos son idénticos. `diff -u original.txt revisado.txt` muestra diferencias de texto. En ambos, el código 0 indica igualdad, 1 indica diferencias y los errores usan otros códigos.

Comprobar integridad con SHA-256

```
# macOS: crear y verificar una lista de huellas
shasum -a 256 datos/notas.txt > resultados/notas.sha256
shasum -a 256 -c resultados/notas.sha256

# Linux con herramientas GNU: alternativa equivalente
sha256sum datos/notas.txt > resultados/notas.sha256
sha256sum -c resultados/notas.sha256
```

Desde la carpeta de práctica se ejecuta el par correspondiente al sistema operativo. La verificación debe indicar `OK`. La huella detecta cambios respecto a la referencia; no valida las notas ni la identidad de quien produjo el archivo.

Otros comandos para el trabajo diario

Ejemplo	Uso
<code>cut -d';' -f2,4 datos/notas.txt</code>	Extraer nombre y nota, incluido el encabezado.
<code>tr '[:lower:]' '[:upper:]' < entrada.txt > salida.txt</code>	Convertir letras según la configuración regional.
<code>mv -i anterior.txt nuevo.txt</code>	Mover o renombrar con confirmación si el destino existe.
<code>rm -i copia.txt</code>	Eliminar con confirmación; normalmente no pasa por la papelera.

Entrada y destino deben ser distintos. Debe evitarse `sort archivo.txt > archivo.txt`: el shell puede vaciar el archivo antes de que `sort` lo lea. El resultado se guarda con otro nombre y se revisa antes de reemplazar el original.

Práctica integradora: revisar antes de analizar

Pedro inspecciona los datos y Julia revisa la evidencia de los hallazgos.

La práctica utiliza el archivo de notas. Pedro ejecuta las verificaciones y guarda las salidas; Julia repite los comandos desde la carpeta de práctica y contrasta los resultados. Después intercambian los papeles.

E1-E2 · UBICAR E INSPECCIONAR

Reconocer el archivo

- ¿Desde qué carpeta deben ejecutarse los ejemplos?
- ¿Cuál es el separador y cuántas filas de datos hay?
- ¿Qué evidencia distingue una nota vacía de una línea vacía?

E3-E4 · CONSULTAR Y RESUMIR

Explicar los conteos

- Localizar las apariciones de Marta con sus números de línea.
- Obtener las frecuencias por grupo y explicar por qué A aparece tres veces.
- Calcular el promedio de las notas presentes en el rango 0 a 5. ¿Por qué aún contiene un duplicado?

E5-E6 · VERIFICAR Y CONSERVAR

Proponer una corrección revisable

- Guardar los identificadores repetidos en resultados/duplicados.txt.
- Documentar la nota vacía y la nota fuera de rango sin inventar valores de reemplazo.
- Crear una copia de trabajo y proponer cómo tratar cada hallazgo.
- Comparar la copia revisada con el original y comprobar la huella del original.

Entrega de la práctica. La entrega incluye los comandos usados, las salidas obtenidas y una breve decisión para cada problema. Una tercera persona debe poder repetir la revisión sin adivinar rutas, separadores ni criterios de validez.

Control para la revisión: 6 filas de datos; Marta en las líneas 4 y 6; nota vacía en la línea 5; valor 6.0 en la línea 7; identificador 3 repetido. Si se elimina solo la fila idéntica duplicada y se excluyen la nota vacía y la nota fuera de rango, quedan 3 notas y su promedio es aproximadamente 4.033. Esa exclusión es una decisión de la práctica, no una regla universal.

Elegir el comando según la pregunta

Un recordatorio para seguir practicando.

Pregunta	Herramientas
¿Cuál es la ubicación actual y qué archivos hay?	<code>pwd</code> , <code>ls -lah</code> , <code>find</code>
¿Qué tipo de archivo es y cuánto ocupa?	<code>file</code> , <code>ls -lh</code> , <code>du -sh</code>
¿Qué hay al principio o al final?	<code>head</code> , <code>tail</code> , <code>less</code>
¿Qué líneas contienen un texto?	<code>grep -n</code> ; <code>-F</code> para texto literal
¿Cuántas líneas o bytes hay?	<code>wc -l</code> , <code>wc -c</code>
¿Cómo se seleccionan campos y se aplican condiciones?	<code>awk</code> , <code>cut</code>
¿Qué se repite y con qué frecuencia?	<code>sort</code> , <code>uniq -d</code> , <code>uniq -c</code>
¿Cambié el contenido de un archivo?	<code>diff -u</code> , <code>cmp</code> , SHA-256

Atajos y ayuda local

Acción	Tecla o comando
Autocompletar rutas y comandos	Tab
Recuperar comandos anteriores	Flechas arriba y abajo
Buscar en el historial	Ctrl + R
Interrumpir un proceso	Ctrl + C
Limpiar la vista de la terminal	Ctrl + L
Consultar el manual	<code>man awk</code> , <code>man grep</code> , <code>man wc</code>
Consultar cómo se resuelve un comando	<code>command -v awk</code>

Cuando una salida no coincide

1. Comprobar la ubicación con `pwd` y el nombre con `ls` .
2. Revisar el encabezado, el separador y los finales de línea.
3. Confirmar si el conteo incluye encabezados, duplicados o campos vacíos.
4. Ejecutar cada etapa de una tubería por separado para localizar la diferencia.

`grep` termina con código 0 si encontró coincidencias y con 1 si no encontró ninguna; un error usa un código distinto. Por eso `grep patrón archivo && otro_comando` solo ejecuta el segundo comando cuando hay coincidencias.

Material de práctica · Octubre de 2026. Ejemplos para archivos de texto simples; las comprobaciones deben adaptarse al formato y a las reglas del conjunto de datos.

Ubicación y archivos

100 comandos de uso frecuente en Linux · Opciones y ejemplos

Selección práctica de uso frecuente, sin un orden estadístico de popularidad. Las opciones corresponden principalmente a GNU/Linux y Bash; algunas herramientas requieren instalación. Las acciones y los formatos se distinguen de las opciones cuando corresponde.

N.º · Comando / función	Opciones, argumentos y ejemplo
01 · <code>pwd</code> Mostrar la ruta actual.	-L: ruta lógica; -P: resolver enlaces. <code>pwd -P</code>
02 · <code>ls</code> Listar archivos.	-l: detalles; -a: ocultos; -h: tamaños legibles; -t: fecha. <code>ls -lah</code>
03 · <code>cd</code> Cambiar de directorio (shell).	-P: ruta física; ../: nivel superior; -: ubicación anterior. <code>cd ..</code>
04 · <code>mkdir</code> Crear directorios.	-p: crear padres; -m: establecer permisos. <code>mkdir -p proyecto/datos</code>
05 · <code>touch</code> Crear un archivo vacío o actualizar fechas.	-a: acceso; -m: modificación; -c: no crear. <code>touch notas.txt</code>
06 · <code>cp</code> Copiar archivos y directorios.	-r: recursivo; -a: preservar atributos; -i: confirmar reemplazo. <code>cp -ai datos respaldo</code>
07 · <code>mv</code> Mover o renombrar.	-i: confirmar reemplazo; -n: no sobrescribir; -v: mostrar acciones. <code>mv -i viejo.txt nuevo.txt</code>
08 · <code>rm</code> Eliminar archivos sin papelera.	-i: confirmar; -r: directorios; -f: sin confirmación. <code>rm -i copia.txt</code>
09 · <code>rmdir</code> Eliminar directorios vacíos.	-p: eliminar también padres vacíos; -v: mostrar acciones. <code>rmdir temporal</code>
10 · <code>ln</code> Crear enlaces.	-s: simbólico; -i: confirmar reemplazo; -v: mostrar acciones. <code>ln -s datos acceso_datos</code>

Las rutas son ejemplos. Las operaciones de copia, movimiento y eliminación modifican archivos; las opciones -i permiten confirmar reemplazos o eliminaciones.

Exploración y metadatos

100 comandos de uso frecuente en Linux · Opciones y ejemplos

N.º · Comando / función	Opciones, argumentos y ejemplo
11 · find Buscar por atributos y nombre.	-type f: archivos; -name: patrón; -mtime: días; -size: tamaño. <pre>find . -type f -name '*.csv'</pre>
12 · locate Buscar rutas en un índice; puede estar desactualizado.	-i: ignorar mayúsculas; -b: nombre base; -l N: límite. <pre>locate -i -l 20 informe</pre>
13 · file Identificar el tipo de contenido.	-b: sin nombre; --mime: tipo MIME; -L: seguir enlaces. <pre>file --mime datos.csv</pre>
14 · stat Mostrar metadatos (GNU).	-c: formato; -f: sistema de archivos; -L: seguir enlaces. <pre>stat -c '%n %s bytes' datos.csv</pre>
15 · du Medir espacio ocupado.	-s: total; -h: legible; --max-depth=N: profundidad. <pre>du -sh datos</pre>
16 · df Consultar espacio disponible.	-h: legible; -T: tipo; -i: inodos. <pre>df -hT .</pre>
17 · lsblk Listar dispositivos de bloques.	-f: sistemas de archivos; -o: columnas; -p: rutas completas. <pre>lsblk -f</pre>
18 · readlink Consultar el destino de un enlace.	-f: resolver ruta; -e: exigir que exista toda la ruta. <pre>readlink -f acceso_datos</pre>
19 · basename Extraer el nombre final de una ruta.	-a: varias rutas; -s SUFIJO: quitar sufijo. <pre>basename -s .csv /datos/ventas.csv</pre>
20 · dirname Extraer la parte de directorio.	Argumento: una o varias rutas; -z: terminación NUL. <pre>dirname /datos/ventas.csv</pre>

locate consulta una base de datos actualizada por updatedb. Su disponibilidad y sus opciones dependen de la implementación. Los dispositivos no se modifican con los ejemplos de consulta.

Lectura y búsqueda de texto

100 comandos de uso frecuente en Linux · Opciones y ejemplos

N.º · Comando / función	Opciones, argumentos y ejemplo
21 · cat Mostrar o concatenar archivos.	-n: numerar; -b: numerar no vacías; -A: caracteres no visibles. <pre>cat -n notas.txt</pre>
22 · less Explorar texto por páginas.	-N: números; -S: no ajustar líneas; -i: búsqueda flexible. <pre>less -N proceso.log</pre>
23 · head Mostrar el inicio.	-n N: líneas; -c N: bytes. <pre>head -n 20 datos.csv</pre>
24 · tail Mostrar el final o seguir cambios.	-n N: líneas; -f: seguir; -F: seguir nombre y reintentar. <pre>tail -F proceso.log</pre>
25 · nl Numerar líneas.	-ba: todas; -w N: ancho; -s TEXTO: separador. <pre>nl -ba notas.txt</pre>
26 · wc Contar contenido.	-l: saltos de línea; -w: palabras; -c: bytes; -m: caracteres. <pre>wc -l datos.csv</pre>
27 · grep Buscar líneas coincidentes.	-n: línea; -i: sin distinción; -v: excluir; -E: regex; -F: literal; -r: recursivo. <pre>grep -n -i 'error' proceso.log</pre>
28 · awk Filtrar campos y calcular.	-F: separador; -v: variable; -f: programa desde archivo. <pre>awk -F',' 'NR>1 {print \$2}' datos.txt</pre>
29 · sed Transformar texto por reglas.	-n: suprimir impresión automática; -E: regex extendida; -e: expresión. <pre>sed -n '10,20p' notas.txt</pre>
30 · cut Extraer campos o posiciones.	-d: delimitador; -f: campos; -c: posiciones de caracteres. <pre>cut -d',' -f1,3 datos.txt</pre>

AWK y cut separan campos simples; no interpretan por sí solos todas las reglas de CSV. wc -l cuenta saltos de línea y grep -c, líneas coincidentes.

Transformación y verificación

100 comandos de uso frecuente en Linux · Opciones y ejemplos

N.º · Comando / función	Opciones, argumentos y ejemplo
31 · <code>sort</code> Ordenar líneas.	-n: numérico; -r: inverso; -k: clave; -t: separador; -u: únicas. <pre>sort -t';' -k4,4n datos_sin_cabecera.txt</pre>
32 · <code>uniq</code> Comparar líneas consecutivas.	-c: frecuencia; -d: repetidas; -u: no repetidas. <pre>sort nombres.txt uniq -c</pre>
33 · <code>tr</code> Traducir o eliminar caracteres.	-d: eliminar; -s: comprimir repeticiones; -c: complementar conjunto. <pre>tr -d '\r' < entrada.txt > salida.txt</pre>
34 · <code>paste</code> Unir líneas de archivos en columnas.	-d: separadores; -s: unir líneas de cada archivo. <pre>paste -d';' nombres.txt notas.txt</pre>
35 · <code>join</code> Unir archivos ordenados por una clave.	-t: separador; -1 N y -2 N: campo de cada entrada; -a 1: incluir no emparejadas. <pre>join -t';' a_ordenado.txt b_ordenado.txt</pre>
36 · <code>split</code> Dividir un archivo.	-l N: líneas; -b N: bytes; -d: sufijos numéricos. <pre>split -l 1000 -d datos.txt parte_</pre>
37 · <code>diff</code> Comparar texto.	-u: contexto unificado; -r: recursivo; -q: indicar solo diferencias. <pre>diff -u original.txt revisado.txt</pre>
38 · <code>cmp</code> Comparar bytes.	-s: silencioso; -l: listar diferencias; -n N: límite. <pre>cmp original.bin copia.bin</pre>
39 · <code>comm</code> Comparar listas ordenadas con la misma configuración regional.	-1, -2, -3: ocultar cada columna de salida. <pre>comm -12 lista_a.txt lista_b.txt</pre>
40 · <code>sha256sum</code> Calcular o verificar huellas SHA-256.	-c: verificar lista; --status: solo código de salida; -b: modo binario. <pre>sha256sum archivo.zip</pre>

join y comm requieren entradas ordenadas de forma compatible. Los ejemplos usan archivos preparados para esa condición; sort aplicado a datos con encabezado también ordena el encabezado.

Shell y ejecución

100 comandos de uso frecuente en Linux · Opciones y ejemplos

N.º · Comando / función	Opciones, argumentos y ejemplo
41 · <code>printf</code> Imprimir con formato.	%s: texto; %d: entero; %f: decimal; \n: salto. Son formatos, no opciones. <pre>printf '%s\n' 'Revisión terminada'</pre>
42 · <code>echo</code> Imprimir argumentos.	-n: omitir salto final (Bash); para escapes portables se prefiere printf. <pre>echo 'Proceso terminado'</pre>
43 · <code>tee</code> Mostrar salida y guardarla.	-a: agregar; -i: ignorar interrupción SIGINT. <pre>wc -l datos.txt tee conteo.txt</pre>
44 · <code>xargs</code> Construir llamadas a partir de la entrada.	-0: entrada NUL; -n N: argumentos por llamada; -P N: procesos; -r: no ejecutar sin entrada. <pre>find . -name '*.txt' -print0 xargs -0 -r wc -l</pre>
45 · <code>history</code> Consultar el historial de Bash.	N: últimas entradas; -a: guardar nuevas; -r: leer historial. <pre>history 20</pre>
46 · <code>alias</code> Definir abreviaturas de Bash.	-p: listar; nombre='comando': definir. <pre>alias ll='ls -lah'</pre>
47 · <code>export</code> Exportar variables en Bash.	-p: listar; -n: retirar atributo de exportación. <pre>export EDITOR=nano</pre>
48 · <code>env</code> Consultar o ajustar el entorno de un proceso.	-i: entorno vacío; -u NOMBRE: omitir variable. <pre>env -u DEBUG programa</pre>
49 · <code>which</code> Buscar ejecutables en PATH.	-a: todas las coincidencias; no describe todos los elementos del shell. <pre>which -a python3</pre>
50 · <code>command</code> Consultar o ejecutar desde el shell.	-v: resolución; -V: descripción; -p: PATH estándar. <pre>command -v awk</pre>

Las opciones de los comandos internos se refieren a Bash. Las tuberías y las redirecciones pertenecen al shell. La sesión actual determina el alcance de alias y export.

Ayuda, identidad y permisos

100 comandos de uso frecuente en Linux · Opciones y ejemplos

N.º · Comando / función	Opciones, argumentos y ejemplo
51 · <code>man</code> Consultar manuales.	-k: buscar temas; -f: descripción; N: sección del manual. <pre>man 1 ls</pre>
52 · <code>apropos</code> Buscar por descripción en manuales.	-i: sin distinción; -e: coincidencia exacta; -s N: sección. <pre>apropos -s 1 archive</pre>
53 · <code>id</code> Consultar identidad y grupos.	-u: UID; -g: GID principal; -G: grupos; -n: nombres. <pre>id -un</pre>
54 · <code>whoami</code> Mostrar el usuario efectivo.	Sin opciones habituales; --help: ayuda; --version: versión. <pre>whoami</pre>
55 · <code>who</code> Listar sesiones conectadas.	-a: información ampliada; -b: último arranque; -q: conteo. <pre>who -b</pre>
56 · <code>chmod</code> Cambiar permisos.	-R: recursivo; -v: informar; u+x: añadir ejecución al propietario. <pre>chmod u+x script.sh</pre>
57 · <code>chown</code> Cambiar propietario y grupo.	-R: recursivo; -h: enlace mismo; --from: propietario previo. <pre>sudo chown ana:datos informe.txt</pre>
58 · <code>chgrp</code> Cambiar grupo de archivos.	-R: recursivo; -h: enlace mismo; -v: informar. <pre>chgrp datos informe.txt</pre>
59 · <code>umask</code> Consultar o fijar la máscara de creación (Bash).	-S: forma simbólica; -p: forma reutilizable; 027: ejemplo de máscara. <pre>umask -S</pre>
60 · <code>sudo</code> Ejecutar con privilegios autorizados.	-u: usuario destino; -l: permisos; -k: invalidar credenciales en caché. <pre>sudo -l</pre>

Cambiar propietarios puede requerir privilegios. `chmod -R`, `chown -R` y `chgrp -R` afectan recursivamente al contenido. La máscara `umask` limita los permisos al crear archivos.

Procesos y trabajos

100 comandos de uso frecuente en Linux · Opciones y ejemplos

N.º · Comando / función	Opciones, argumentos y ejemplo
61 · ps Obtener una instantánea de procesos.	-e: todos; -f: formato completo; -o: columnas; -p: PID. <pre>ps -eo pid,comm,%cpu,%mem</pre>
62 · top Monitorizar procesos.	-d N: intervalo; -p PID: filtrar; -b: modo lote; -n N: iteraciones. <pre>top -b -n 1</pre>
63 · htop Explorar procesos de forma interactiva; requiere instalación.	-d N: décimas de segundo; -p: PID; -u: usuario. <pre>htop -u ana</pre>
64 · pgrep Buscar identificadores de procesos.	-a: línea completa; -f: buscar en argumentos; -u: usuario; -x: nombre exacto. <pre>pgrep -a -x python3</pre>
65 · kill Enviar una señal a un PID.	-TERM: terminar ordenadamente; -KILL: forzar; -l: listar señales. <pre>kill -TERM 1234</pre>
66 · pkill Enviar señales a procesos coincidentes.	-x: nombre exacto; -f: argumentos; -u: usuario; -TERM: señal. <pre>pkill -TERM -x programa_prueba</pre>
67 · jobs Listar trabajos de la sesión de Bash.	-l: incluir PID; -p: solo PID; -r: en ejecución; -s: detenidos. <pre>jobs -l</pre>
68 · fg Pasar un trabajo al primer plano.	%N: número mostrado por jobs; no lleva opciones de uso común. <pre>fg %1</pre>
69 · bg Reanudar un trabajo detenido en segundo plano.	%N: número mostrado por jobs; no lleva opciones de uso común. <pre>bg %1</pre>
70 · nohup Ejecutar ignorando la señal de cierre SIGHUP.	Sin opciones operativas habituales; & es un operador del shell. <pre>nohup bash tarea.sh > tarea.log 2>&1 &</pre>

Los PID, nombres y números de trabajo son ejemplos. TERM permite la finalización ordenada; KILL no permite limpieza. jobs, fg y bg operan sobre trabajos de la sesión actual.

Sistema y servicios

100 comandos de uso frecuente en Linux · Opciones y ejemplos

N.º · Comando / función	Opciones, argumentos y ejemplo
71 · <code>uname</code> Consultar información del sistema.	-a: toda; -r: kernel; -m: arquitectura. <pre>uname -a</pre>
72 · <code>hostname</code> Consultar el nombre del equipo.	-f: nombre completo; -l: direcciones disponibles (Linux). <pre>hostname -I</pre>
73 · <code>uptime</code> Consultar tiempo activo y carga.	-p: duración legible; -s: fecha de arranque. <pre>uptime -p</pre>
74 · <code>free</code> Consultar memoria y swap.	-h: legible; -m: MiB; -s N: intervalo en segundos. <pre>free -h</pre>
75 · <code>date</code> Consultar o formatear fechas (GNU).	-u: UTC; -d: interpretar fecha; +FORMATO: salida. <pre>date -u '+%Y-%m-%d %H:%M:%S UTC'</pre>
76 · <code>watch</code> Repetir una consulta.	-n N: intervalo; -d: resaltar cambios; -t: sin encabezado. <pre>watch -n 2 'df -h .'</pre>
77 · <code>systemctl</code> Consultar y administrar servicios systemd.	status/start/stop/restart: acciones; --user: usuario; --failed: fallidos. <pre>systemctl --failed</pre>
78 · <code>journalctl</code> Consultar registros de systemd.	-u: unidad; -b: arranque; -f: seguir; -n N: últimas; --since: desde fecha. <pre>journalctl -b -n 50</pre>
79 · <code>dmesg</code> Consultar mensajes del kernel; puede requerir privilegios.	-H: legible; -T: fechas; -w: seguir; --level: nivel. <pre>dmesg --level=err,warn</pre>
80 · <code>crontab</code> Consultar o editar tareas cron del usuario.	-l: listar; -e: editar; -r: eliminar la tabla completa. <pre>crontab -l</pre>

`systemctl` y `journalctl` requieren `systemd`. Las unidades disponibles dependen del sistema. `crontab -r` elimina la programación del usuario; `-l` solo la consulta.

Red y acceso remoto

100 comandos de uso frecuente en Linux · Opciones y ejemplos

N.º · Comando / función	Opciones, argumentos y ejemplo
81 · ip Consultar interfaces y rutas.	-br: breve; -c: color; addr/link/route: objetos; show: consulta. <pre>ip -br addr</pre>
82 · ss Consultar sockets.	-t: TCP; -u: UDP; -l: escucha; -n: numérico; -p: procesos. <pre>ss -tuln</pre>
83 · ping Comprobar respuestas ICMP.	-c N: cantidad; -W N: espera; -I: interfaz u origen. <pre>ping -c 4 example.com</pre>
84 · traceroute Consultar saltos hacia un destino.	-n: sin DNS; -m N: máximo de saltos; -I: ICMP. <pre>traceroute -n example.com</pre>
85 · dig Consultar DNS.	+short: breve; @SERVIDOR: DNS; A/MX/TXT: tipo de registro. <pre>dig +short example.com A</pre>
86 · curl Transferir datos por URL.	-I: cabeceras; -L: redirecciones; -o: destino; -f: fallar en HTTP erróneo; -sS: silencioso con errores. <pre>curl -I https://example.com</pre>
87 · wget Descargar archivos.	-O: destino; -c: continuar; -q: silencioso; --spider: comprobar recurso. <pre>wget --spider https://example.com</pre>
88 · ssh Iniciar una sesión remota segura.	-p: puerto; -i: clave privada; -L: reenvío local; -v: diagnóstico. <pre>ssh -p 22 usuario@servidor</pre>
89 · scp Copiar mediante SSH.	-r: directorios; -P: puerto; -i: clave privada; -p: preservar fechas y modos. <pre>scp informe.txt usuario@servidor:/tmp/</pre>
90 · rsync Sincronizar archivos.	-a: archivo; -v: detalle; -h: legible; -n: simulación; --delete: eliminar extras del destino. <pre>rsync -avhn origen/ destino/</pre>

Los destinos remotos son ilustrativos. La ausencia de respuesta ICMP no demuestra que un servicio esté caído. En rsync, -n simula; --delete puede borrar archivos del destino.

Compresión, paquetes y edición

100 comandos de uso frecuente en Linux · Opciones y ejemplos

N.º · Comando / función	Opciones, argumentos y ejemplo
91 · tar Agrupar o extraer archivos.	-c: crear; -x: extraer; -t: listar; -f: archivo; -z: gzip; -J: xz. <pre>tar -czf respaldo.tar.gz datos/</pre>
92 · gzip Comprimir o descomprimir gzip.	-d: descomprimir; -k: conservar original; -c: salida estándar; -t: integridad. <pre>gzip -k datos.txt</pre>
93 · gunzip Descomprimir gzip.	-k: conservar comprimido; -c: salida estándar; -t: comprobar integridad. <pre>gunzip -k datos.txt.gz</pre>
94 · zip Crear archivos ZIP.	-r: recursivo; -u: actualizar; -q: silencioso. <pre>zip -r respaldo.zip datos/</pre>
95 · unzip Consultar o extraer ZIP.	-l: listar; -t: integridad; -d: destino; -n: no sobrescribir. <pre>unzip -l respaldo.zip</pre>
96 · apt Gestionar paquetes en Debian y Ubuntu.	update/install/remove/search/show: acciones; -s: simular cambios. <pre>apt show curl</pre>
97 · dnf Gestionar paquetes en Fedora y sistemas compatibles.	install/remove/upgrade/search/info: acciones; --assumeno: responder no. <pre>dnf info curl</pre>
98 · nano Editar texto.	-l: números de línea; -B: respaldo; -v: solo lectura. <pre>nano -l notas.txt</pre>
99 · vim Editar texto con modos.	-R: solo lectura; -d: diferencias; +N: iniciar en línea N. <pre>vim -R notas.txt</pre>
100 · tmux Mantener sesiones de terminal.	new -s: crear; attach -t: conectar; ls: listar sesiones. <pre>tmux new -s trabajo</pre>

apt y dnf corresponden a familias distintas de distribuciones. Los comandos de consulta no instalan paquetes. gzip y gunzip conservan el original en los ejemplos gracias a -k.

Interpretación y referencias

Consulta de detalles y diferencias entre implementaciones.

La lista reúne 100 comandos distintos por nombre, agrupados por finalidad. Incluye herramientas externas y comandos internos del shell. No representa una clasificación medida de frecuencia de uso.

ARCHIVO , N , PID , los usuarios y los destinos representan valores que deben adaptarse al caso. Las opciones no se combinan automáticamente: cada ejemplo utiliza solo las necesarias. En comandos internos de Bash, help nombre ofrece ayuda local; para herramientas externas se utiliza man nombre .

Familia	Documentación de referencia
Archivos, texto y huellas	Manual de GNU Coreutils
Shell y comandos internos	Manual de GNU Bash
Búsqueda y procesamiento	GNU grep · GNU AWK · GNU sed · GNU Findutils
Procesos y red	ps · ip · ss
Servicios	systemctl · journalctl
Transferencias y archivos	curl · OpenSSH · rsync · GNU tar
Paquetes y editores	Manuales locales: man apt , man dnf , man nano , man vim y man tmux .

Las versiones instaladas y las políticas de acceso determinan qué opciones y operaciones están disponibles. La guía principal mantiene ejemplos de texto sencillos compatibles con Linux y macOS; este anexo se centra en Linux.